

# Communication entre Micro-services

Synchrone vs Asynchrone

OpenFeign

RabbitMQ



Préparée par :

Ons Ben Amor

# Les deux modes de communication

## SYNCHRON E

### Communication bloquante

Le service appelant attend une réponse immédiate avant de poursuivre son exécution. L'appelant est bloqué pendant toute la durée de l'échange.

#### Technologie : OpenFeign

Client HTTP déclaratif Java. Les interfaces annotées @FeignClient remplacent les appels HTTP manuels, avec découverte via Eureka.

## ASYNCHRON E

### Communication non bloquante

Le service expéditeur publie un message sans attendre de réponse. Les services fonctionnent indépendamment et en parallèle.

#### Technologie : RabbitMQ

Broker de messages AMQP. Les producteurs publient dans des queues, les consommateurs lisent de manière différée.

# Tableau comparatif

| Critère          | Synchrone — OpenFeign                   | Asynchrone — RabbitMQ                |
|------------------|-----------------------------------------|--------------------------------------|
| Nature           | Appel bloquant — attend la réponse      | Envoi non bloquant — pas d'attente   |
| Protocole        | HTTP / REST                             | AMQP (port 5672)                     |
| Couplage         | Fort — disponibilité simultanée requise | Faible — services indépendants       |
| Tolérance pannes | Faible — échec si service indispo       | Élevée — messages conservés en queue |
| Réponse          | Immédiate et directe                    | Différée — aucun retour direct       |
| Scalabilité      | Limitée par disponibilité du service    | Élevée — consommateurs parallèles    |
| Complexité       | Simple — interface Java + annotations   | Modérée — broker, queues, converter  |
| Ordre messages   | Non applicable                          | Garanti (FIFO)                       |
| Cas d'usage      | Données en temps réel                   | Notifications, emails, favoris       |

# Quand choisir quel mode ?

## Choisir Synchrone

### Réponse immédiate nécessaire

- Récupérer des données en temps réel
- Interroger un service pour afficher un résultat
- Interactions simples et directes
- Ex : afficher la liste des jobs du MS Job

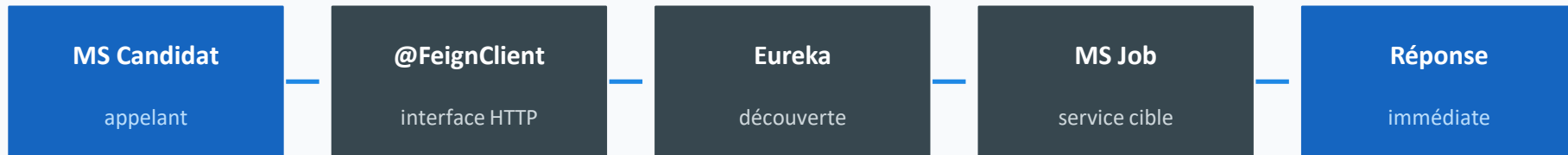
## Choisir Asynchrone

### Pas de réponse directe requise

- Tâches longues ou déconnectées
- Envoi d'emails ou notifications
- Traitement à fort volume
- Ex : ajouter un job aux favoris du candidat

# Flux de communication — Synchrone (OpenFeign)

Le service appelant est bloqué jusqu'à réception de la réponse



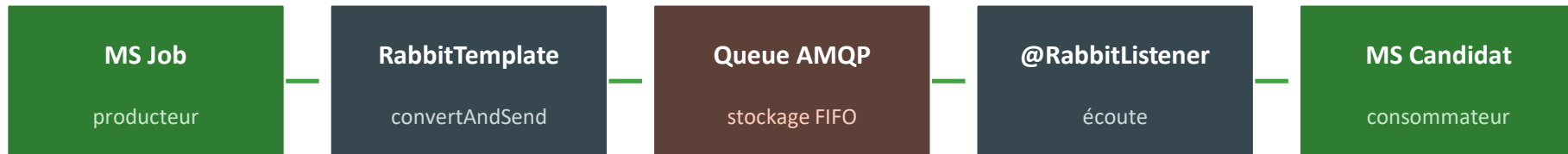
← Réponse retournée à l'appelant (bloquant)

```
@FeignClient(name = "job-s")
public interface JobClient {

    @GetMapping("/jobs")
    List<Job>
    getAllJobs();
}
```

# Flux de communication — Asynchrone (RabbitMQ)

Le producteur n'attend pas — le message est stocké dans la queue jusqu'à consommation



Pas de réponse directe — communication découplée

Producteur (MS Job)

```
rabbitTemplate.  
    convertAndSend(  
  
    "jobQueue", jobDTO);
```

Consommateur (MS Candidat)

```
@RabbitListener(queues =  
  
    "jobQueue")  
public void  
    receiveJob(JobDTO dto) {}
```

# RabbitMQ : Création & configuration de la queue

## Qui déclare la queue ?



Imagine une boîte aux lettres 📧

- Consumer = la personne qui reçoit
- Producer = la personne qui envoie

Qui crée la boîte aux lettres ?

La personne qui reçoit  
(Consumer)

Pas celui qui envoie



### Traduction en RabbitMQ



Queue =  
boîte aux lettres



Consumer =  
propriétaire



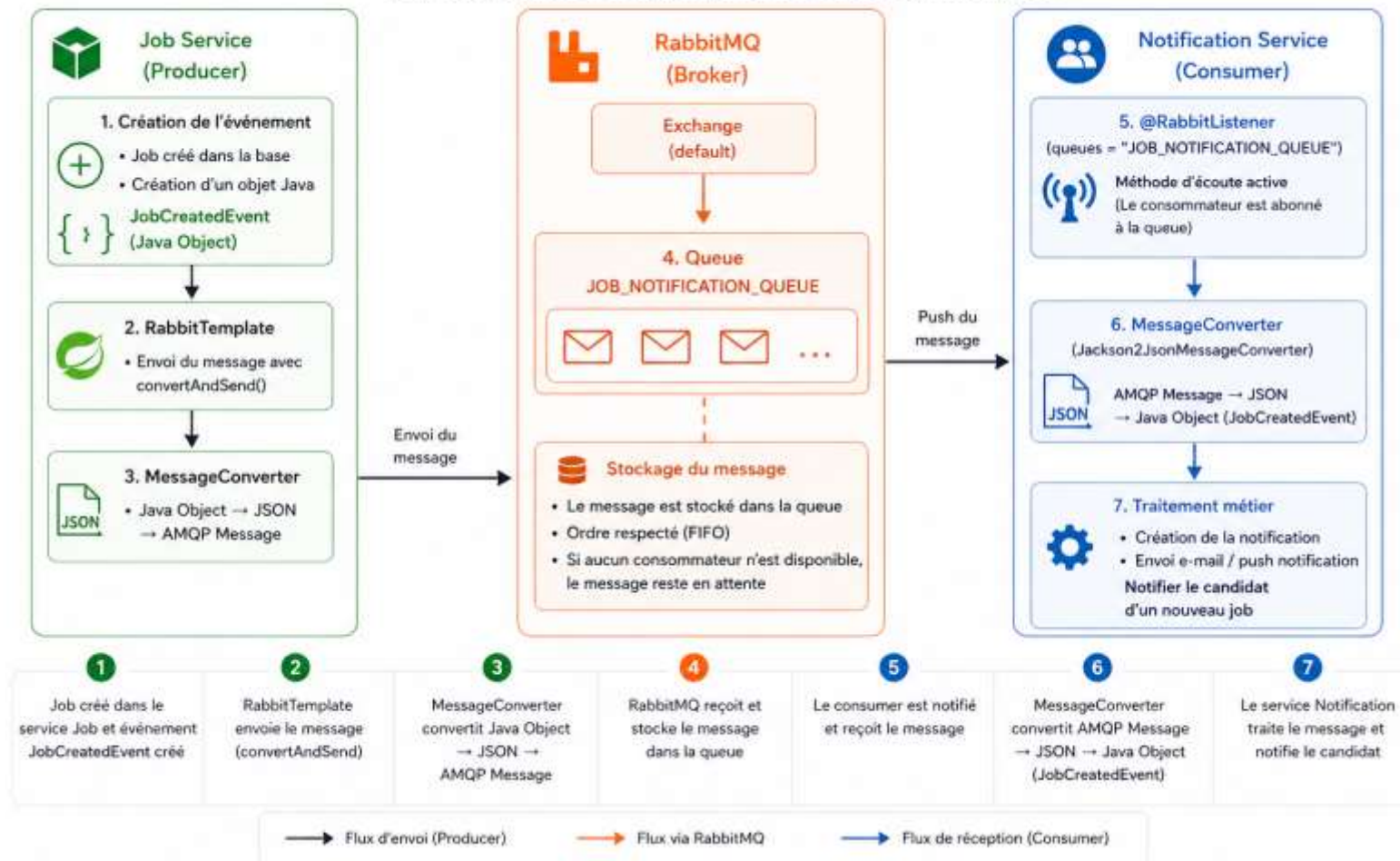
Producer =  
expéditeur

### RÈGLE CLÉ

Le consumer déclare la queue.  
Le producer envoie simplement  
des messages à cette queue.

# SCÉNARIO : NOTIFICATION LORS DE L'AJOUT D'UN JOB

Objectif : Notifier le candidat lorsqu'un nouveau job est ajouté





# Points clés à retenir

- 01 Synchrones = bloquant, réponse immédiate. Asynchrones = non bloquant, découplé.
- 02 OpenFeign simplifie les appels REST inter-services via des interfaces annotées.
- 03 RabbitMQ garantit la livraison des messages même si le consommateur est hors-ligne.
- 04 La queue est déclarée côté consommateur (principe de responsabilité unique).
- 05 Le choix dépend du besoin : temps réel → sync / traitement différé → async.